



| RN SHETTY TRUST®
RNS INSTITUTE OF TECHNOLOGY

Autonomous Institution Affiliated to Visvesvaraya Technological University, Belagavi
Approved by AICTE, New Delhi, Accredited by NAAC with 'A+' Grade
Channasandra, Dr. Vishnuvardhan Road, Bengaluru - 560 098
Ph: (080) 28611880 / 1; www.rnsit.ac.in



DEPARTMENT OF CSE(AI&ML)

PYTHON PROGRAMMING LABORATORY

LAB Manual

(**BPRLC205B**)

(As per Autonomous 2024 Scheme Course type- IPCC)

Compiled by

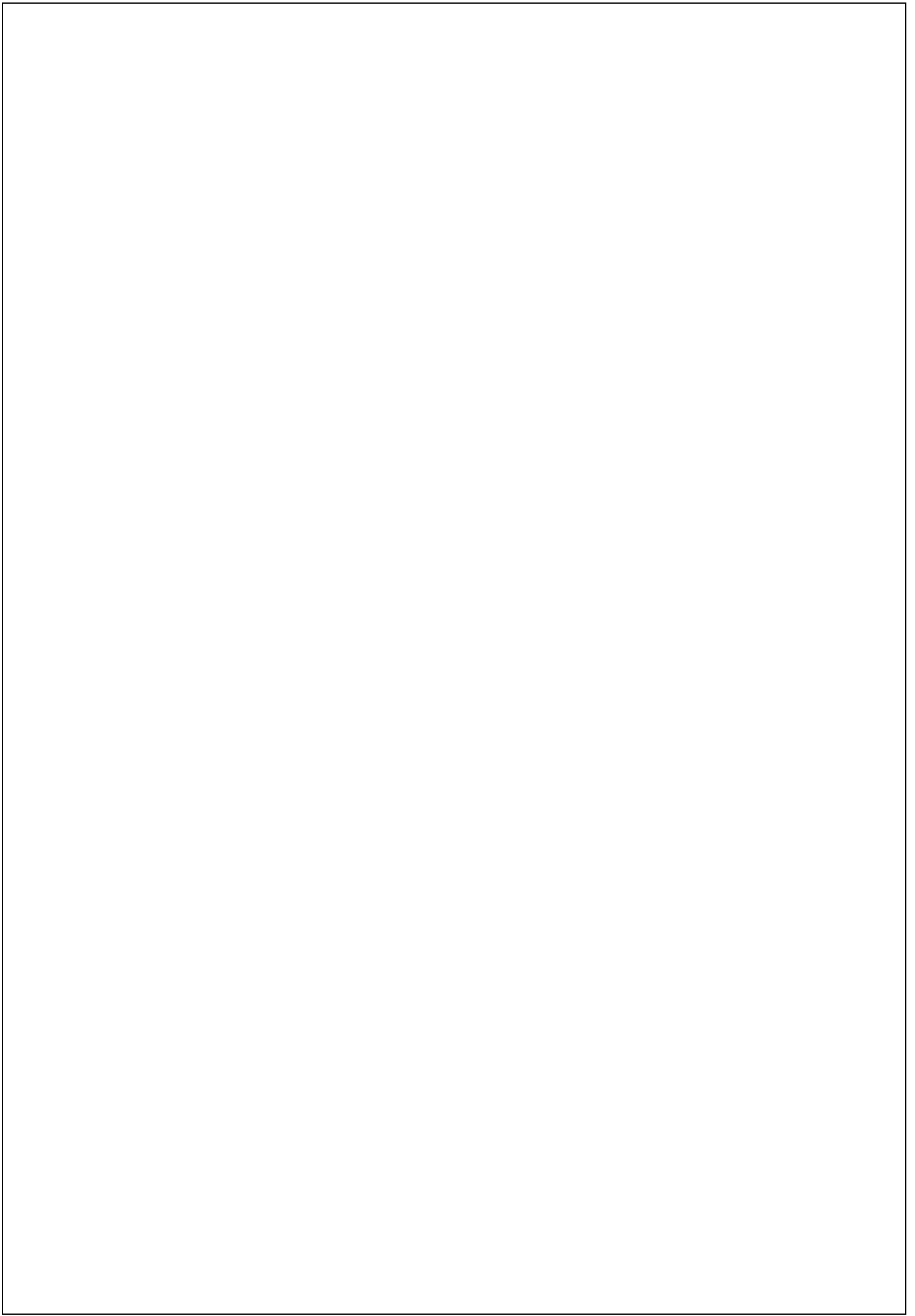
Department of CSE(AI&ML)

RNS Institute of Technology

Bengaluru-98

Name: _____

USN: _____



PYTHON PROGRAMMING LABORATORY (BPLCK105B) INTERNAL**EVALUATION SHEET****EVALUATION(MAXMARKS20)**

TEST	REGULAR EVALUATION	RECORD	TOTAL MARKS
A	B		A+B+C
8	7	5	20

R1:REGULARLAB EVALUATION WRITE UP RUBRIC(MAXMARKS7)

Sl. No.	Parameters	Good	Average	Needs improvement
a.	Understanding of problem (3M)	Clear understanding of problem statement while designing and implementing (3)	Problem statement is understood clearly but few mistakes while designing and implementing(2)	Problem statement is not clearly understood while designing (1)
b.	Write-up (2M)	Write the program neatly, demonstrating the concepts of Data Structure and algorithms using C Language (2)	Moderate understanding of algorithms (1)	Problem statement is not clearly understood while writing a program (1)
c.	Execution and Result (2M)	Program handles all possible conditions(2)	Demonstrates some of the conditions defined (1)	Documentation does not take care all conditions (1)

R2:REGULARLAB EVALUATION VIVA RUBRIC(MAXMARKS7)

Sl. No.	Parameter	Excellent	Good	Average	Needs Improvement
a.	Conceptual understanding (10marks)	Answers 80% of the viva questions asked (10)	Answers 60% of the viva questions asked (7)	Answers 30% of the viva question asked (4)	Unable to relate the concepts (1)

R3:REGULARLAB PROGRAM EXECUTION RUBRIC(MAXMARKS 10)

Sl. No.	Parameters	Excellent	Good	Needs Improvement
a.	Design, implementation and demonstration (5M)	Program follows syntax and semantics of C programming language .Demonstrate the complete knowledge of the program written (5)	Program has few logical errors, moderately demonstrates all possible concepts implemented in Program	Documentation does not take care the given conditions(2)
b.	Result & documentation (5M)	All test cases are successful, all errors are debugged with own practical knowledge and clear documentation according to the guidelines (5)	Moderately debugs the experiment, few test case are unsuccessful and Partial documentation (4)	Test cases are not taken care, unable to debug the errors And no proper documentation(1)

R4:RECORD EVALUATION RUBRIC(MAXMARKS5)

Sl. No.	Parameter	Excellent	Good	Average	Needs Improvement
a.	Documentation (20M)	Meticulous record writing required Complete Program with comments and test cases as per the guidelines mentioned (20)	Write up contains Program and test cases with no comments (18)	Write up contains only program(15)	Program written with few mistakes. (10)

A. TEST/LAB INTERNALS MARKS(MAX MARKS 8)

TEST#	Writeup 10M	Execution 30M	Viva 10M	Teacher Signature	Total 40M	Average 50M	Final 20
TEST-1						<u>50M</u>	<u>20M</u>
TEST-2							

B. REGULAR LAB EVALUATION(MAX MARKS 5)

Sl.#	Date of Execution	Additional programs	Write up (4)	Execn(4)	Viv a (4)	Total 12	Teacher Signature
1.a							
1.b							
1.c							
2.a							
2.b							
3.a							
3.b							
4.a							
5.a							
6.a							
7.a							
7.b							
8.a							
8.b							
9.a							
10.a							
11.a							
12.a							
Total Marks		<u>12</u>	<u>12</u>			<u>12</u>	

EVALUATION SHEET

Course Title: Python Programming

Courser Code: BPRLC205B

Final Marks obtained from test (08)	08	
Regular evaluation (12)	12	
Total Marks obtained	20	

Lab in-charge:

Head of the Department:

PREFACE

We have developed this comprehensive laboratory manual on Python **Programming (BPRLC205B)** with the primary objectives: Familiarize the students with computer hardware , concept of naming the program files, storing, compilation, execution and debugging.

This material provides the students annex posture to problem solving approaches and solution number of problems using Python programming Language. The problems discussed in this manual comprises of an algorithm, programming solution, alternative logic and extensive test cases. Viva questions, frequently appeared examination questions and practicing programming problems constitute an indispensable part of this material.

Our profound and sincere efforts will be fruitful only when students acquire the extensive knowledge by reading this manual and apply the concepts learnt.

Department of CSE(AI&ML)

TABLE OF CONTENTS			
SL. NO.	CONTENTS	PAGE NO.	CO'S
	INTRODUCTION TO COMPUTERS HARDWARE AND PROGRAMMING ENVIRONMENTS		
1.	Computer hardware and programming environment		
1.1	Functional block diagram of computer		
1.2	Operating system and its types		
1.3	Programming environments		
1.3.1	Python Program Execution in Windows Operating System		
1.3.2	Python Program Execution in Linux Operating System		
1.4	Problem solving techniques		
	LABORATORY PROGRAMS		
1	. a. Write a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide. b. Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or no. c. Write a python program to add 10 numbers by inputting each from the keyboard using for loop		CO1
2	a. Develop a program to generate Fibonacci sequence of length (N). Read N from the console. b. Write a python program to create a list and perform the following operations • Inserting an element • Removing an element • Appending an element • Displaying the length of the list • Popping an element • Clearing the list		CO2
3	a. Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages. b. Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with a suitable message.		CO2
4	a. Develop a program to print 10 most frequently appearing words in a text file. [Hint: Use dictionary with distinct words and their frequency of occurrences. Sort the dictionary in the reverse order of frequency and display dictionary slice of first 10 items]		CO3

Python Programming (BPRLC205B)

5	a. Develop a program to read 6 subject marks from the keyboard for a student. Generate a report that displays the marks from the highest to the lowest score attained by the student. [Read the marks into a 1-Dimesional array and sort using the Bubble Sort technique].		CO2
6	a. Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods strip(), len(), list methods sort(), append(), and file methods open(), readlines(), and write()].		CO3
7	a. Write a function named DivExp which takes TWO parameters a, b and returns a value c ($c=a/b$). Write suitable assertion for $a>0$ in function DivExp and raise an exception for when $b=0$. Develop a suitable program which reads two values from the console and calls a function DivExp. b. Write a python program to create a class called time with members hours, minutes, and seconds. Store user entered time and print the total number of seconds.		CO5
8	a. Define a function that takes TWO objects representing complex numbers and returns a new complex number with the sum of two complex numbers. Define a suitable class,,Complex“ to represent the complex number. Develop a program to read N ($N \geq 2$) complex numbers and to compute the addition of N complex numbers. b. Write a python program to implement inheritance: <pre> graph TD Employee --> Clerk Employee --> Software_Engineer[Software Engineer] Software_Engineer --> Team_Leader[Team Leader] </pre>		CO5
9	a. Text Analysis Tool: Build a tool that analyses a paragraph: frequency of each word,longest word, number of sentences, etc..		CO2
10	a. Develop Data Summary Generator: Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column		CO3
11	a. Develop Student Grade Tracker: Accept multiple students“ names and marks. Store them in a list of tuples or dictionaries. Display summary reports (average, topper, etc.).		CO2
12	a. Develop a program to display contents of a folder recursively (Directory) having subfolders and files (name and type).		CO2

SYALLABUS**SEMESTER-II****PYTHON PROGRAMMING**
(Effective from the academic year 2024-2025)

Course Code – BPLCK205B	CIE Marks -50
Number of Contact Hours/Week -3:0:2:0	SEE Marks -50
Total Number of Lab Contact Hours -20	Exam Hours-03

Course Learning Objectives:

This course will enable students to:

- Learn the syntax and semantics of the Python programming language.
- Illustrate the process of structuring the data using lists, tuples, sets and Dictionary
- Illustrate the working of strings and automating tasks using Regular expressions
- Demonstrate the use of built-in functions to navigate the file system and work with various documents like PDF, word and others
- Enable debugging and implement exception handling in python

Experiments (12 Lab slots)**1. Basics -conditionals statements and iteration statements**

- Write a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide).
- Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or no.
- Write a python program to add 10 numbers by inputting each from the keyboard using for loop Variables

2. List and function

- Develop a program to generate Fibonacci sequence of length (N). Read N from the console.
- Write a python program to create a list and perform the following operations
 - Inserting an element
 - Removing an element
 - Appending an element
 - Displaying the length of the list
 - Popping an element
 - Clearing the list

3. Statistics

- Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages.
- Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with a suitable message.

4. File handling

- Develop a program to print 10 most frequently appearing words in a text file. [Hint: Use dictionary with distinct words and their frequency of occurrences. Sort the dictionary in the reverse order of frequency and display dictionary slice of first 10 items]

5. Sorting

- Develop a program to read 6 subject marks from the keyboard for a student. Generate a report that displays the marks from the highest to the lowest score attained by the student. [Read the marks into a 1-Dimensional array and sort using the Bubble Sort technique].

6. Text Handling

- Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods strip(), len(), list methods sort(), append(), and file methods open(), readlines(), and write()].

7 Error handling

- Write a function named DivExp which takes TWO parameters a, b and returns a value c ($c=a/b$). Write suitable assertion for $a>0$ in function DivExp and raise an exception for when $b=0$. Develop a suitable program which reads two values from the console and calls a function DivExp.
- Write a python program to create a class called time with members hours, minutes, and seconds. Store user entered time and print the total number of seconds.

8 Class and Objects

- Define a function that takes TWO objects representing complex numbers and returns a new complex number with the sum of two complex numbers. Define a suitable class, "Complex" to represent the complex number. Develop a program to read N ($N \geq 2$) complex numbers and to compute the addition of N complex numbers.
- Write a python program to implement inheritance:

9. Text Analysis Tool:

- Build a tool that analyses a paragraph: frequency of each word, longest word, number of sentences, etc..

10. Handling Csv file

- Develop Data Summary Generator: Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column

11. Tuples and dictionaries

- Develop Student Grade Tracker: Accept multiple students' names and marks. Store them in a list of tuples or dictionaries. Display summary reports (average, topper, etc.).

12. File handling

- Develop a program to display contents of a folder recursively (Directory) having subfolders and files (name and type).

Assessment Type	Maximum Marks	Minimum Passing Marks	Evaluation Details
CIE- IA Tests (A)	20	-	Two tests each of 50 marks shall be conducted. Average of Two Internal Assessment Tests shall be scaled down to 20 marks.
Continuous and Comprehensive Evaluation (B)	10	-	Any one Assessment methods as per 24RNBE4.2 scaled to 10 marks.
Total CIE Theory (A+B)	30	12	Marks of Tests and Assignments awarded for a maximum of 30.
CIE Practical	12	-	Marks awarded for Weekly Conduction of Experiments and Submission of Laboratory records, scaled to 12 marks.
CIE Practical Test	8	-	Average of marks of Two tests, each conducted for 100 marks covering all experiments, scaled to 8.

Python Programming (BPRLC205B)

Total CIE Practical (C)	20	8	Marks of Experiments, Record and Test awarded for a maximum of 20 marks.
CIE: Theory + Practical (A + B + C)	50	20	CIE Evaluation is done for a maximum of 50 marks.
SEE (D)	50	18	SEE is a theory exam, conducted for 100 marks and scaled down to 50 marks. Exam duration shall be 3 hours.
CIE+SEE (A+B+C+D)	100	40	Final marks shall be the sum of marks scored in Components A, B, C and D

The minimum marks to be secured in CIE is 20 (40%) out of allotted 50 marks to appear for SEE. The laboratory component of the IPCC shall be for CIE only. However, in SEE, the questions from the laboratory component shall be included in their respective modules only.

Introduction

1. Familiarization with computer hardware and programming environment, concept of naming the program files, storing, compilation, execution and debugging. Taking any simple C- code.

1.1 Description about Functional block diagram of Computer

A computer is an electronic device, which mainly performs the four functions as **reading, processing, displaying** and **storing** on data. Fig. 1 portrays various functional units of computer system. These functions of a computer system can be carried out by using the three main units namely input unit, system unit and output unit. The block diagram of a computer system is as follows:

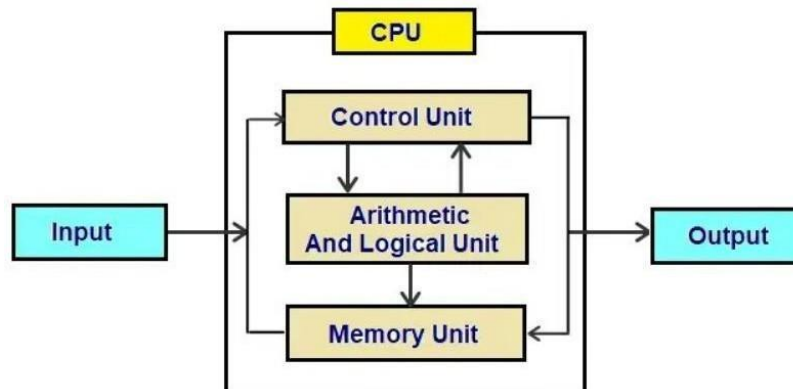


Fig 1: Block Diagram of a Computer

Notations:

- Data and Results flow
- Control instructions to other units from control unit
- Instructions from memory unit to control unit



System Unit or Central Processing Unit (CPU): is commonly known as “processor” that executes the instructions of a computer program. It has Control Unit (CU) and Arithmetic & Logical Unit (ALU).

Input unit: is used to enter data and information into a computer. The devices like keyboard, mouse and scanner are commonly used input devices.

- A keyboard is used to enter alphanumeric characters and symbols.
- The mouse is used to pick or select a command from the monitor screen.
- A scanner is used to scan an image or read a barcode and so on.

Arithmetic and Logic Unit (ALU): is a digital circuit that performs arithmetic (Add, Sub, Multiplication, Division) and logical (AND, OR, NOT) operations.

Control unit (CU): is the circuitry that controls the flow of information through the processor and coordinates the activities of the other units within the processor.

Functions of Control unit

- Accessing data and instructions from memory unit.
- Interpreting instructions.
- Controlling input and output units.
- Overall supervision of a computer system.

Memory Unit (MU): is the unit where all the input data and results are stored either temporarily or permanently. The memory of a computer has two types:

- **Main Memory /Primary Memory units**
 - Random Access Memory(RAM)
 - Read Only memory(ROM)
- **Secondary Memory /Auxiliary Memory**

Output Unit: It is used to display or print results from a computer. Monitor, printer and plotters are commonly used output devices.

Bus: A bus is a collection of wires that carries data/instructions. It connects physical components such as printed circuit boards, CPU, memory, peripherals etc., for sharing of information and communication with one another.

Types of Buses:

System Buses: The system buses are used to transfer the data and instructions between Main memory (Random Access Memory) and CPU. Table 1 lists the different types of system buses.

Table 1: Types of System Buses

Data Bus	Address Bus	Control Bus
It is used to transfer the data between Processor, Memory and I/O devices	It is used to transfer the addresses of data and Instructions stored in Memory.	It is used to transfer the control signals between CPU, Memory and I/O devices.
Bidirectional in nature	Unidirectional in nature	Unidirectional or Bidirectional in nature

I/O Buses: The buses which are used to connect all I/O devices with CPU and Memory are called I/O buses. Table 2 shows three types of I/O buses.

Table 2: Types of I/O buses

PCI Bus	ISA Bus	USB Bus
PCI stands for <i>Peripheral Component Interconnect</i>	ISA stands for <i>Industry Standard Architecture</i>	USB stands for <i>Universal Serial Bus</i>
The motherboard will be having 3 or 4 PCI connectors, so that we can insert various chips.	This is simple and slowest bus used in IBM PCs	It helps to connect various I/O devices like keyboard mouse, pen drives, printer, etc.
Fastest and presently more powerful bus	Oldest, simple and slowest bus	New and widely used bus

Mother Board: Mother Board is a set of Integrated Chips (ICs) which are designed to work together. It controls the flow of data/instructions within computer. It is the main board on which other hardware components are connected to enable the computer system to work as an integrated unit. It consists of sockets, slots, power connectors and bus.

Chip Sets: Chip set is the set of integrated chips that are designed to work together. These set of chips controls the flow of information on computer. Fig. 2 depicts chip sets. The chips may be controllers for memory, cache, hard drive, keyboard and peripherals.



Fig 2: Chip Sets

1.2 Operating System and its types

An Operating System (OS) is system software that **controls** and **supervises** the **hardware components** of a computer system, and it provides the services to computer users. Also called as **Resource Manager** that manages the resources such as CPU, Memory, I/O devices, Job/Task/Process etc., a computer cannot run without OS. Fig. 3 depicts architecture of operating system. The major function of OS includes: **CPU Management, Memory Management, File Management, Device Management, Process/Task/Job Management and Security Management.**

Some of the examples of Operating Systems are Windows, UNIX, XENIX, Macintosh OS, Fedora, and Android, etc.

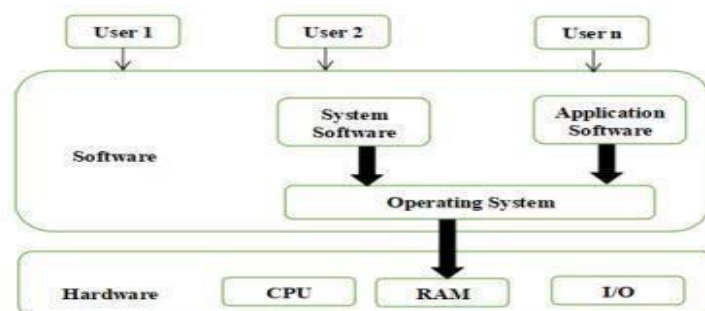


Fig 3: Operating System

1.3 Problem Solving Techniques

There are three approaches to problem solving ,namely :Algorithm, Flow chart and Pseudocode

1.3.1 Algorithm: The algorithm is a *step-by-step procedure* to be followed in solving a problem. It provides a scheme to solve a particular problem in *finite number of unambiguous steps*. It helps in implementing the solution of a problem using any of the *programming languages*.

In order to qualify as an algorithm a sequence of instructions must possess the following characteristics:

Definiteness: Instructions must be *precise and unambiguous* i.e. each and every instruction should be clear and should have only one meaning.

Finiteness: *Note* single instruction must be *repeated infinitely* .i.e., each instruction should be performed in finite time.

Termination : At some finite step the algorithm should terminate.

Example: To compute the Area of Rectangle

ALGORITHM :AREA_of_RECTANGLE

[This algorithm takes length and breadth, the sides of the rectangle as input and computes the area of rectangle using the formula ***area=length * breadth***. Finally, it prints the area of rectangle]

Step1:[Initialize] Start

Step2:[Input the sides of Rectangle] Read **length, breadth**

Step3:[Compute the area of rectangle]

Area=length*breadth

Step 4:[Display the Area]

Print Area

Step5:[Finished]

Stop

ALGORITHM:SIMPLE INTEREST

Step1:[Initialize] Start

Step2:[Input the sides of Rectangle] Read **P,T and R**

Step3:[Compute the area of rectangle]

SI=P*T*R/100

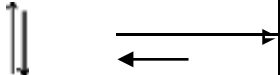
Step4:[Display the Area] Print **SI**

Step5:[Finished]

Stop

1.3.2 Flowchart: A flowchart is a graphical or symbolic representation of an algorithm. They are basically used to design and develop complex programs to help the users to visualize the logic of the program so that they can gain a better understanding of the program and find flaws, bottlenecks, and other less-obvious features within it. The following Table 3 shows the symbols used in flowchart along with its descriptions. Fig. 7 depicts the flowchart to compute area of rectangle.

Table3:Commonflowchart symbols

Symbol	Name	Description
	Oval	Represents the starting and terminal point
	Rectangle	Represents the process steps defined in algorithm
	Parallelogram	Indicate the reading operation used for input/output or data or information from/to any device
	Rhombus	Indicates the decisions(questions)and consequently the branch points or the paths to be followed based on the Result of the question
	Arrows	Shows the flowchart direction and connects the various flow chart symbols
	Small circle	Shows the continuation from one Point in the process flow to another
	Hexagon	Represents Looping structures
	Predefined process	Indicates Subroutines

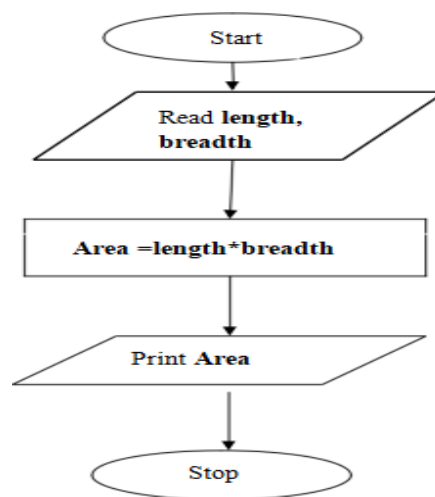


Fig 4: To compute the area of rectangle

Python Programming (BPRLC205B)

Pseudo code: A compact and informal high-level description of an algorithm that uses the structural conventions of a programming language. Details such as keywords, variable declarations and system-specific code are not used while writing pseudo code.

Example: To compute the area of Rectangle

Begin

 Input length, breadth

 Area=length*breadth

 Print Area

End

SAMPLE PYTHON PROGRAM

/*program to print area of a triangle*/

l = 10

b = 20

area=0.5*b*h

print("Area of triangle is :", area)

PROGRAMS

1 a. Write a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide.)

Purpose:

To demonstrate input () and print() statements and perform basic arithmetic operations.

Input:

Two numbers (num1, num2), and a choice (1-Add, 2-Subtract, 3-Multiply, 4-Divide).

Expected Output:

Display the result of the selected operation with a suitable message.

Program

```
# Simple calculator program

# Read two numbers from the keyboard
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

# Show menu
print("\nChoose an operation:")
print("1 - Add")
print("2 - Subtract")
print("3 - Multiply")
print("4 - Divide")

choice = int(input("Enter your choice (1-4): "))

# Perform operation based on choice
if choice == 1:
    print("Result:", num1 + num2)
elif choice == 2:
    print("Result:", num1 - num2)
elif choice == 3:
    print("Result:", num1 * num2)
elif choice == 4:
    if num2 != 0:
        print("Result:", num1 / num2)
    else:
        print("Error: Division by zero is not allowed!")
else:
    print("Invalid choice! Please select between 1 and 4.")
```

Output

Case1:

Enter first number: 10

Enter second number: 5

Choose an operation:

1 - Add

2 - Subtract

3 - Multiply

4 - Divide

Enter your choice (1-4): 1

Result: 15.0

Case2:

Enter first number: 10

Enter second number: 5

Choose an operation:

1 - Add

2 - Subtract

3 - Multiply

4 - Divide

Enter your choice (1-4): 1

Result: 15.0

Case3:

Enter first number: 12

Enter second number: 3

Choose an operation:

1 - Add

2 - Subtract

3 - Multiply

4 - Divide

Enter your choice (1-4): 7

Invalid choice! Please select between 1 and 4.

1b. Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or not.

Purpose : To demonstrate if else conditional statements and importing modules

Input: Person Name and person year of birth

Expected Output: Display whether the person is a senior citizen or not

Program

```
from datetime import date
perName = input("Enter the name of the person: ")
perDOB = int(input("Enter his year of birth: "))

curYear = date.today().year
perAge = curYear - perDOB

if perAge > 60:
    print(perName, "aged", perAge, "years is a Senior Citizen.")
else:
    print(perName, "aged", perAge, "years is not a Senior Citizen.")
```

Output

Enter the name of the person : “Anish” Enter his
year of birth : 1975
Anish aged 49 years is not a Senior Citizen.

- Enter the name of the person : “Anu” Enter his
year of birth : 1952
Anish aged 72 years is a Senior Citizen

1 c. Write a python program to add 10 numbers by inputting each from the keyboard using for loop.

Purpose:

To demonstrate the use of for loop, input(), and print() by reading 10 numbers from the keyboard and calculating their sum.

Input:

10 numbers (entered one by one from the keyboard).

Expected Output:

Display the sum of all 10 numbers with a suitable message.

```
# Program to add 10 numbers using for loop
```

```
total = 0
```

```
print("Enter 10 numbers:")
```

```
for i in range(10):
```

```
    num = float(input(f"Enter number {i+1}: "))
```

```
    total += num
```

```
print("\nThe sum of the 10 numbers is:", total)
```

OUTPUT:

CASE1:

Enter 10 numbers:

Enter number 1: 1

Enter number 2: 2

Enter number 3: 3

Enter number 4: 4

Enter number 5: 5

Enter number 6: 6

Enter number 7: 7

Enter number 8: 8

Enter number 9: 9

Enter number 10: 10

The sum of the 10 numbers is: 55.0

Case 2: Mix of positive and negative

Enter 10 numbers:

Enter number 1: 10

Enter number 2: -5

Enter number 3: 3

Enter number 4: -2

Python Programming (BPRLC205B)

Enter number 5: 8

Enter number 6: 4

Enter number 7: -1

Enter number 8: 6

Enter number 9: -3

Enter number 10: 5

The sum of the 10 numbers is: 25.0

2.a. Develop a program to generate Fibonacci sequence of length (N). Read N from the console.

Purpose: To demonstrate the working of loop

Input: Enter the value of n

Expected Output: Fibonacci sequence of length (N).

Program

```
num = int(input("Enter the Fibonacci sequence length to be generated : "))
firstTerm = 0
secondTerm = 1
print("The Fibonacci series with", num, "terms is :")
print(firstTerm, secondTerm, end=" ")
for i in range(2, num):
    curTerm = firstTerm + secondTerm
    print(curTerm, end=" ")
    firstTerm = secondTerm
    secondTerm = curTerm
```

Output

Enter the Fibonacci sequence length to be generated : 12 The
Fibonacci series with 12 terms is :
0 1 1 2 3 5 8 13 21 34 55 8

2.b Write a python program to create a list and perform the following operations

- ***Inserting an element***
- ***Removing an element***
- ***Appending an element***
- ***Displaying the length of the list***
- ***Popping an element***
- ***Clearing the list***

Purpose:

To demonstrate basic list operations in Python such as inserting, removing, appending, finding length, popping, and clearing.

Input:

A list created by the user and elements to perform operations on.

Expected Output:

Display the list after each operation and show the effect of list functions.

Program:

```
# Program to demonstrate list operations

# Creating a list
my_list = [10, 20, 30, 40, 50]
print("Initial List:", my_list)

# Inserting an element at position 2
my_list.insert(2, 25)
print("After Inserting 25 at index 2:", my_list)

# Removing an element (first occurrence of 40)
my_list.remove(40)
print("After Removing 40:", my_list)

# Appending an element
my_list.append(60)
print("After Appending 60:", my_list)

# Displaying the length of the list
print("Length of the List:", len(my_list))

# Popping an element (removes last element)
popped = my_list.pop()
print("After Popping Last Element:", my_list)
print("Popped Element:", popped)

# Clearing the list
my_list.clear()
print("After Clearing the List:", my_list)
```

Output

Initial List: [10, 20, 30, 40, 50]

After Inserting 25 at index 2: [10, 20, 25, 30, 40, 50]

After Removing 40: [10, 20, 25, 30, 50]

After Appending 60: [10, 20, 25, 30, 50, 60]

Length of the List: 6

After Popping Last Element: [10, 20, 25, 30, 50]

Popped Element: 60

After Clearing the List: []

3.a. Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages.

Purpose: To demonstrates the concept of List.

Input: Input number from the console N

Expected Output: To compute and display mean, variance and standard deviation with suitable messages

Program

```
from math import sqrt
myList = []
num = int(input("Enter the number of elements in your list : "))
for i in range(num):
    val = int(input("Enter the element : "))
    myList.append(val)
print('The length of list1 is', len(myList))
print('List Contents', myList)
total = 0
for elem in myList:
    total += elem
mean = total / num
total = 0
for elem in myList:
    total += (elem - mean) * (elem - mean)
variance = total / num
stdDev = sqrt(variance)
print("Mean =", mean)
print("Variance =", variance)
print("Standard Deviation =", "%.2f"%stdDev)
```

Output

```
Enter the number of elements in your list : 5
Enter the element : 2
Enter the element : 3
Enter the element : 4
Enter the element : 5
Enter the element : 6
The length of list1 is 5
List Contents [2,3,4,5,6]
Mean = 4.0
Variance = 2.0
Standard Deviation = 1.41
```

3.b Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with suitable message.

Purpose: To demonstrate the concept of set

Input: num

Expected Output: print the frequency of each digit with suitable message.

Program

```
num = input("Enter a number : ")
print("The number entered is :", num)
uniqDig = set(num)
for elem in uniqDig:
    print(elem, "occurs", num.count(elem), "times")
```

Output

```
Enter a number : 12111
The number entered is : 12111
2 occurs 1 times
1 occurs 4 times
```

4.a Develop a program to print 10 most frequently appearing words in a text file.
[Hint: Use dictionary with distinct words and their frequency of occurrences. Sort the dictionary in the reverse order of frequency and display dictionary slice of first 10 items]

Purpose: To demonstrate the concept of dictionaries

Input: File

Expected Output: Frequency of each word in file

Program

```
import sys
import string
import os.path
fname = input("Enter the filename : ")
#sample file text.txt also provided
if not os.path.isfile(fname):
    print("File", fname, "doesn't exists")
    sys.exit(0)
infile = open(fname, "r")
filecontents = ""
for line in infile:
    for ch in line:
        if ch not in string.punctuation:
            filecontents = filecontents + ch
        else:
            filecontents = filecontents + ' '
#replace punctuations and \n with space
wordFreq = {}
wordList = filecontents.split()
#Calculate word Frequency for
word in wordList:
    if word not in wordFreq.keys():
        wordFreq[word] = 1
    else:
        wordFreq[word] += 1
#Sort Dictionary based on values in descending order
sortedWordFreq = sorted(wordFreq.items(), key=lambda x:x[1], reverse=True )
for i in range(10):
```

```
print(sortedWordFreq[i][0], "occurs", sortedWordFreq[i][1], "times")  
sortedWordFreq = sorted(wordFreq.items(), key=lambda x:x[1], reverse=True )
```

Output

```
Enter the filename : romeo.txt is occurs 3 times  
the occurs 3 times  
and occurs 3 times  
sun occurs 2 times  
But occurs 1 times  
soft occurs 1 times  
what occurs 1 times  
light occurs 1 times  
through occurs 1times  
yonder occurs 1times
```

5. Develop a program to read 6 subject marks from the keyboard for a student. Generate a report that displays the marks from the highest to the lowest score attained by the student. [Read the marks into a 1-Dimensional array and sort using the Bubble Sort technique].

Purpose:

To read 6 subject marks of a student, store them in a 1-Dimensional array (list), and display the marks in descending order using the **Bubble Sort technique**.

Input:

6 subject marks entered from the keyboard.

Expected Output:

Marks displayed from **highest to lowest** after applying Bubble Sort.

Program

```
# Program to sort 6 subject marks in descending order using Bubble Sort

# Reading 6 marks into a list

marks = []

print("Enter marks for 6 subjects:")

for i in range(6):

    m = int(input(f"Enter mark {i+1}: "))

    marks.append(m)

print("\nOriginal Marks:", marks)

# Bubble Sort (Descending order)

n = len(marks)

for i in range(n):

    for j in range(0, n-i-1):

        if marks[j] < marks[j+1]: # swap if next is greater

            marks[j], marks[j+1] = marks[j+1], marks[j]

# Display result

print("Marks from Highest to Lowest:", marks)
```

Output

Case 1: Random Marks

Input:

56, 78, 45, 89, 92, 67

Output:

Original Marks: [56, 78, 45, 89, 92, 67]

Marks from Highest to Lowest: [92, 89, 78, 67, 56, 45]

Case 2: Ascending Order Marks

Input:

10, 20, 30, 40, 50, 60

Output:

Original Marks: [10, 20, 30, 40, 50, 60]

Marks from Highest to Lowest: [60, 50, 40, 30, 20, 10]

6. Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods `strip()`, `len()`, list methods `sort()`, `append()`, and file methods `open()`, `readlines()`, and `write()`].

Purpose:

To read the contents of a text file, sort the lines, and write the sorted lines into a new text file using string methods (`strip()`, `len()`), list methods (`sort()`, `append()`), and file methods (`open()`, `readlines()`, `write()`).

Input:

A text file (`input.txt`) with unsorted contents (lines of text).

Expected Output:

A new text file (`output.txt`) with contents sorted in ascending order.

Program

```
# Program to sort the contents of a text file and write to another file

# Open input file and read lines
with open("input.txt", "r") as infile:
    lines = infile.readlines()

# Remove newline characters and extra spaces
cleaned_lines = []
for line in lines:
    cleaned_lines.append(line.strip())

# Sort the lines
cleaned_lines.sort()

# Write sorted lines to output file
with open("output.txt", "w") as outfile:
    for line in cleaned_lines:
        outfile.write(line + "\n")

print("Sorting complete. Check 'output.txt' for results.")
```

Output

Case 1: Input file (`input.txt`)

Banana
Apple
cherry
mango
grapes

Output file (output.txt)

apple
banana
cherry
grapes
mango

Case 2: Input file (input.txt)

zebra
lion
tiger
elephant
ant

Output file (output.txt)

ant
elephant
lion
tiger

zebra

Case 3: Input file (input.txt) with repeated words

dogcat
dog
elephant
ant

Output file (output.txt)

Ant
cat
dog
Dog
elephant

7.a. Develop a function named DivExp which takes TWO parameters a, b, and returns a value c ($c = a/b$).

Write a suitable assertion for $a > 0$ in the function DivExp and raise an exception for when $b = 0$.

Develop a suitable program that reads two console values and calls the function DivExp.]

Purpose:

To demonstrate how to create a function that performs division, uses assertions to validate input ($a > 0$), and raises an exception when division by zero occurs.

Input:

Two numbers entered from the keyboard:

a (numerator, must be > 0)

b (denominator, must not be 0)

Expected Output:

If valid: Prints $c = a/b$

If $a \leq 0$: Prints an **Assertion Error** message

If $b = 0$: Prints a **ZeroDivisionError** message

If invalid input: Prints **Invalid input** message

Program

```
def DivExp(a, b):
    assert a > 0, "Value of a must be greater than 0"
    if b == 0:
        raise ZeroDivisionError("Division by zero is not allowed!")
    return a / b

def main():
    try:
        a = float(input("Enter value of a: "))
        b = float(input("Enter value of b: "))
        result = DivExp(a, b)
        print(f"Result (c = a/b): {result}")
    except AssertionError as e:
        print("Assertion Error:", e)
    except ZeroDivisionError as e:
        print("Error:", e)
    except ValueError:
        print("Invalid input. Please enter numbers only.")
```

main()

Python Programming (BPRLC205B)

```
● PS C:\Users\Dell\Downloads> python manual.py
Enter value of a: 25
Enter value of b: 5
Result (c = a/b): 5.0
● PS C:\Users\Dell\Downloads> python manual.py
Enter value of a: 0
Enter value of b: 5
Assertion Error: Value of a must be greater than 0
● PS C:\Users\Dell\Downloads> python manual.py
Enter value of a: 50
Enter value of b: 0
Error: Division by zero is not allowed!
● PS C:\Users\Dell\Downloads> python manual.py
Enter value of a: ?
Invalid input. Please enter numbers only.
○ PS C:\Users\Dell\Downloads>
```

7b. Write a python program to create a class called time with members hours, minutes, and seconds.

Store user-entered time and print the total number of seconds.

Purpose:

To demonstrate how to create a Python class with

members hours, minutes, and seconds, take user input, and calculate the total number of seconds.

Input:

hours (integer)

minutes (integer)

seconds (integer)

Expected Output:

Prints the total seconds computed as:

total_seconds = hours*3600 + minutes*60 + seconds

If invalid input: Prints **Invalid input** message

class Time:

def __init__(self, hours, minutes, seconds):

self.hours = hours

self.minutes = minutes

self.seconds = seconds

def total_seconds(self):

return self.hours * 3600 + self.minutes * 60 + self.seconds

def main():

try:

h = int(input("Enter hours: "))

m = int(input("Enter minutes: "))

s = int(input("Enter seconds: "))

t = Time(h, m, s)

print(f"Total seconds = {t.total_seconds()}")

except ValueError:

print("Invalid input. Please enter integers only.")

main()

```
PS C:\Users\De11\Downloads> python manual.py
Enter hours: 5
Enter minutes: 45
Enter seconds: 25
Total seconds = 20725
PS C:\Users\De11\Downloads> █
```

8a Define a function that takes TWO objects representing complex numbers and returns a new complex number with the sum of two complex numbers. Define a suitable class to represent complex numbers and to compute the addition of N complex numbers.

Purpose:

To create a class

representing complex numbers, define a function to add two complex numbers, and compute the sum of N complex numbers.

Input:

- ❖ Real and imaginary parts of two complex numbers for pair addition
- ❖ Number of complex numbers N and their real and imaginary parts for sum of N numbers

Expected Output:

- ❖ Sum of two complex numbers in standard form ($a + bi$)
- ❖ Sum of N complex numbers in standard form

Program

```
class ComplexNumber:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    def add(self, other):
        return ComplexNumber(self.real + other.real, self.imag + other.imag)

    def __str__(self):
        sign = '+' if self.imag >= 0 else '-'
        return f"{self.real} {sign} {abs(self.imag)}i"
```

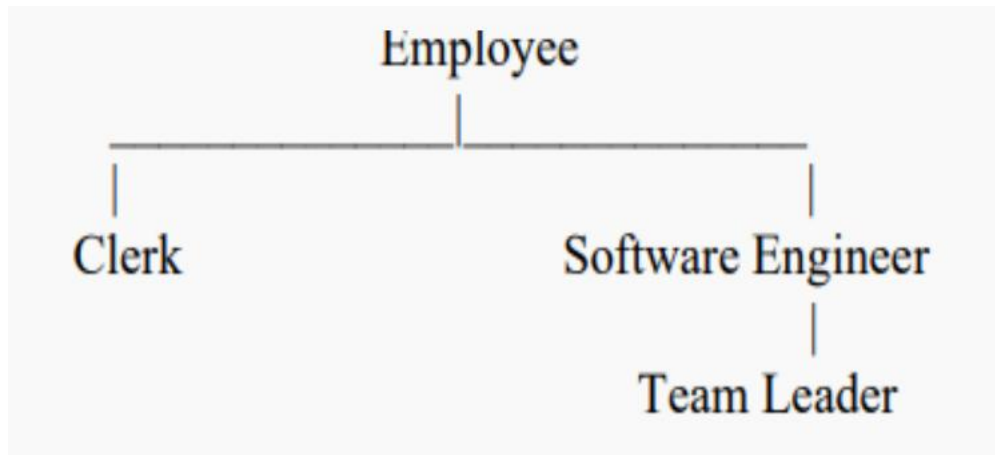
Python Programming (BPRLC205B)

```
def main():  
  
    n = int(input("\nEnter how many complex numbers to add: "))  
    total = ComplexNumber(0, 0)  
    for i in range(n):  
        r = float(input(f"Enter real part of number {i+1}: "))  
  
        im = float(input(f"Enter imaginary part of number {i+1}: "))  
        total = total.add(ComplexNumber(r, im))  
  
    print(f"Sum of {n} complex numbers: {total}")  
  
main()
```

Output

```
Enter two complex numbers  
Enter real part of first number: 5  
Enter imaginary part of first number: 9  
Enter real part of second number: 6  
Enter imaginary part of second number: 8  
Sum of two complex numbers: 11.0 + 17.0i  
  
Enter how many complex numbers to add: 3  
Enter real part of number 1: 2  
Enter imaginary part of number 1: 4  
Enter real part of number 2: 10  
Enter imaginary part of number 2: 9  
Enter real part of number 3: 7  
Enter imaginary part of number 3: 6  
Sum of 3 complex numbers: 19.0 + 19.0i
```

8b Write a python program to implement inheritance:



Purpose:

To demonstrate **class inheritance** in Python by creating a base class Employee and derived classes Clerk, SoftwareEngineer, and TeamLeader. Display details of each type of employee.

Input:

Predefined employee details (name, salary, department/language/team size)

Expected Output:

Display information of each employee including all class-specific attributes.

```
# Base class
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display(self):
        print(f"Employee Name: {self.name}, Salary: {self.salary}")

# Derived class 1
class Clerk(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department

    def display(self):
        print(f"Clerk Name: {self.name}, Salary: {self.salary}, Department: {self.department}")

# Derived class 2
class SoftwareEngineer(Employee):
    def __init__(self, name, salary, language):
        super().__init__(name, salary)
        self.language = language
```

Python Programming (BPRLC205B)

```
def display(self):
    print(f"Software Engineer Name: {self.name}, Salary: {self.salary}, Language: {self.language}")

# Derived class 3 (inherits from SoftwareEngineer)

class TeamLeader(SoftwareEngineer):
    def __init__(self, name, salary, language, team_size):
        super().__init__(name, salary, language)
        self.team_size = team_size

    def display(self):
        print(f"Team Leader Name: {self.name}, Salary: {self.salary}, Language: {self.language}, Team Size: {self.team_size}")

# Main function
def main():
    c = Clerk("John", 25000, "Accounts")
    se = SoftwareEngineer("Alice", 60000, "Python")
    tl = TeamLeader("Bob", 80000, "Java", 5)

    c.display()
    se.display()
    tl.display()

main()
```

Output

```
Clerk Name: John, Salary: 25000, Department: Accounts
Software Engineer Name: Alice, Salary: 60000, Language: Python
Team Leader Name: Bob, Salary: 80000, Language: Java, Team Size: 5
```


9 . Text Analysis Tool: Build a tool that analyses a paragraph: frequency of each word, longest word, number of sentences, etc.

Purpose:

To analyze a paragraph by computing:

- ❖ Number of sentences
- ❖ Number of words
- ❖ Longest word
- ❖ Frequency of each word

Input:

A paragraph entered from the keyboard (can contain multiple sentences with punctuation ., !, ?).

Expected Output:

- ❖ Total number of sentences
- ❖ Total number of words
- ❖ The longest word
- ❖ Frequency of each word (case-insensitive)

Program

```
def text_analysis(paragraph):
    paragraph = paragraph.strip()

    sentences = 0
    for ch in paragraph:
        if ch in ".!?!":
            sentences += 1

    words = paragraph.replace('.', '').replace('!', '').replace('?', '').split()
    freq = {}
    for word in words:
        word_lower = word.lower()
        freq[word_lower] = freq.get(word_lower, 0) + 1

    longest_word = max(words, key=len)

    print("\n--- Text Analysis ---")
    print(f"Number of sentences: {sentences}")
    print(f"Number of words: {len(words)}")
    print(f"Longest word: {longest_word}")
    print("\nWord Frequencies:")
    for word, count in freq.items():
        print(f"{word}: {count}")

def main():
    paragraph = input("Enter a paragraph: ")
    text_analysis(paragraph)

main()
```

```
PS C:\Users\Dell\Downloads> python manual.py
Enter a paragraph: Python is easy to learn. Python is powerful and simple!

--- Text Analysis ---
Number of sentences: 2
Number of words: 10
Longest word: powerful

Word Frequencies:
python: 2
is: 2
easy: 1
to: 1
learn: 1
powerful: 1
and: 1
simple: 1
```

10. Develop Data Summary Generator: Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column.

Purpose:

To read a CSV file, convert it into dictionary form, and allow the user to run **summary queries** (maximum, minimum, average) on any numeric column.

Input:

- ❖ CSV file name (e.g., covid_data.csv or weather.csv)
- ❖ Column name to summarize

Expected Output:

- ❖ Maximum value of the selected column
- ❖ Minimum value of the selected column
- ❖ Average value of the selected column
- ❖ Error message if the column is non-numeric

Program

```
def read_csv_to_dict(filename):
    data = []
    with open(filename, "r") as file:
        lines = file.readlines()
        headers = lines[0].strip().split(",")
        for line in lines[1:]:
            values = line.strip().split(",")
            row_dict = dict(zip(headers, values))
            data.append(row_dict)
    return data, headers

def summary_queries(data, column):
    try:
        values = [float(row[column]) for row in data]
        print(f"Max of {column}: {max(values)}")
        print(f"Min of {column}: {min(values)}")
        print(f"Average of {column}: {sum(values) / len(values):.2f}")
    except ValueError:
        print("Selected column is not numeric.")

def main():
    filename = input("Enter CSV file name: ")
    data, headers = read_csv_to_dict(filename)

    print("Available columns:", headers)
    column = input("Enter column name for summary: ")
    summary_queries(data, column)

main()
```

```
● PS C:\Users\Dell\Downloads> python manual.py
Enter CSV file name: covid_data.csv
Available columns: ['Date', 'Cases', 'Deaths']
Enter column name for summary: Deaths
Max of Deaths: 3.0
Min of Deaths: 1.0
Average of Deaths: 2.00
○ PS C:\Users\Dell\Downloads> █
```

11. Develop Student Grade Tracker: Accept multiple students' names and marks. Store them in a list of tuples or dictionaries. Display summary reports (average, topper, etc.).

Purpose:

To accept multiple students' names and marks, store them in a list of dictionaries, and display a summary report including class average and the topper.

Input:

- ❖ Number of students n
- ❖ Each student's name and marks

Expected Output:

- ❖ Class average
- ❖ Topper's name and marks

Program (using dictionary)

```
def student_grade_tracker():
    students = []
    n = int(input("Enter number of students: "))

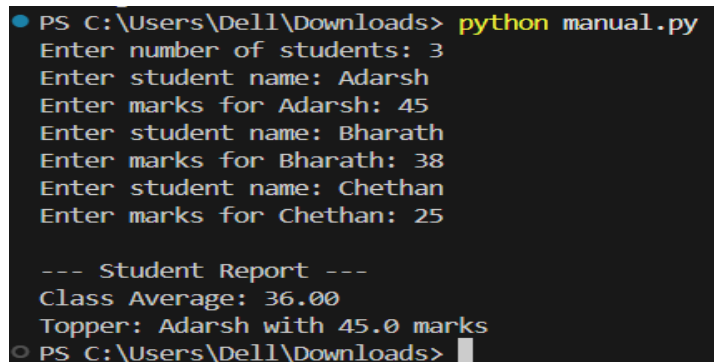
    for _ in range(n):
        name = input("Enter student name: ")
        marks = float(input(f"Enter marks for {name}: "))
        students.append({"name": name, "marks": marks})

    total = sum(s["marks"] for s in students)
    average = total / n

    topper = max(students, key=lambda s: s["marks"])

    print("\n--- Student Report ---")
    print(f"Class Average: {average:.2f}")
    print(f"Topper: {topper['name']} with {topper['marks']} marks")

student_grade_tracker()
```



```
PS C:\Users\Dell\Downloads> python manual.py
Enter number of students: 3
Enter student name: Adarsh
Enter marks for Adarsh: 45
Enter student name: Bharath
Enter marks for Bharath: 38
Enter student name: Chethan
Enter marks for Chethan: 25

--- Student Report ---
Class Average: 36.00
Topper: Adarsh with 45.0 marks
PS C:\Users\Dell\Downloads>
```

Python Programming (BPRLC205B)

Program (using TUPLE)

```
def student_grade_tracker():
    students = []
    n = int(input("Enter number of students: "))

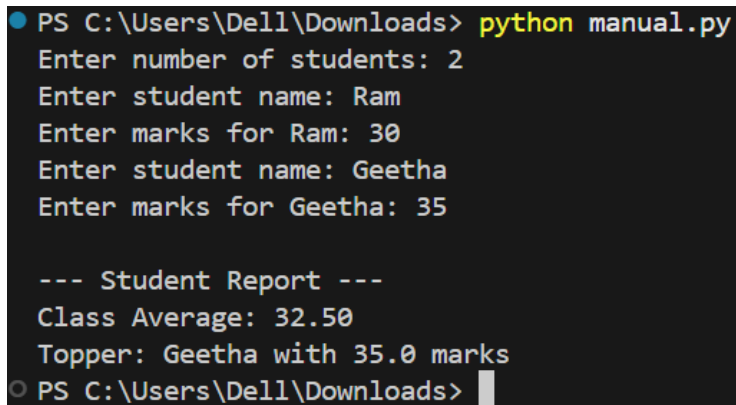
    for _ in range(n):
        name = input("Enter student name: ")
        marks = float(input(f"Enter marks for {name}: "))
        students.append((name, marks))

    total = sum(s[1] for s in students)
    average = total / n

    topper = max(students, key=lambda s: s[1])

    print("\n--- Student Report ---")
    print(f"Class Average: {average:.2f}")
    print(f"Topper: {topper[0]} with {topper[1]} marks")

student_grade_tracker()
```



```
PS C:\Users\Dell\Downloads> python manual.py
Enter number of students: 2
Enter student name: Ram
Enter marks for Ram: 30
Enter student name: Geetha
Enter marks for Geetha: 35

--- Student Report ---
Class Average: 32.50
Topper: Geetha with 35.0 marks
PS C:\Users\Dell\Downloads>
```

12. Develop a program to display contents of a folder recursively (Directory) having sub-folders and files (name and type).

Purpose: To display all files and subfolders within a specified folder path using Python's os module.

Input: Folder path entered by the user

Expected Output:

- ❖ Lists all directories and files under the given folder path
- ❖ Marks directories with [Folder] and files with [File]
- ❖ Prints error message if the folder path is invalid

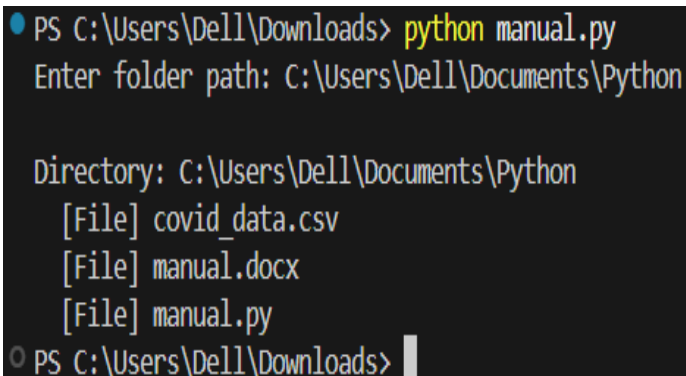
Program

```
import os

def display_folder_contents(path):
    for root, dirs, files in os.walk(path):
        print(f"\nDirectory: {root}")
        for d in dirs:
            print(f" [Folder] {d}")
        for f in files:
            print(f" [File] {f}")

def main():
    folder = input("Enter folder path: ")
    if os.path.exists(folder):
        display_folder_contents(folder)
    else:
        print("Invalid folder path!")

main()
```



```
PS C:\Users\Dell\Downloads> python manual.py
Enter folder path: C:\Users\Dell\Documents\Python

Directory: C:\Users\Dell\Documents\Python
 [File] covid_data.csv
 [File] manual.docx
 [File] manual.py
PS C:\Users\Dell\Downloads>
```

VIVA QUESTIONS:

1. What type of language is python? Programming or scripting?

Python is capable of scripting, but in general sense, it is considered as a general-purpose programming language.

2. Python an interpreted language. Explain.

Ans: An interpreted language is any programming language which is not in machine-level code before runtime. Therefore, Python is an interpreted language.

3. What are the benefits of using Python?

Ans: The benefits of using python are-

Easy to use– Python is a high-level programming language that is easy to use, read, write and Python Programming learn.

Interpreted language– Since python is interpreted language, it executes the code line by line and stops if an error occurs in any line.

Dynamically typed– the developer does not assign data types to variables at the time of coding. It automatically gets assigned during execution.

Free and open-source– Python is free to use and distribute. It is open source.

Extensive support for libraries– Python has vast libraries that contain almost any function needed. It also further provides the facility to import other packages using Python Package Manager(pip).

Portable– Python programs can run on any platform without requiring any change.

The data structures used in python are user friendly.

It provides more functionality with less coding.

4. What are Python namespaces?

Ans: A namespace in python refers to the name which is assigned to each object in python. The objects are variables and functions. As each object is created, its name along with space(the address of the outer function in which the object is), gets created. The namespaces are maintained in python like a dictionary where the key is the namespace and value is the address of the object.

There 4 types of namespace in python-

Built-in namespace– These namespaces contain all the built-in objects in python and are available whenever python is running.

Global namespace– These are namespaces for all the objects created at the level of the main program.

Enclosing namespaces– These namespaces are at the higher level or outer function.

Local namespaces– These namespaces are at the local or inner function

5. What is slicing in Python?

Ans: Slicing is used to access parts of sequences like lists, tuples, and strings. The syntax of slicing is-[start:end:step]. The step can be omitted as well.

When we write [start:end] this returns all the elements of the sequence from the start (inclusive) till the end-1 element.

If the start or end element is negative i, it means the ith element from the end. The step indicates the jump or how many elements have to be skipped.

Eg. if there is a list- [1,2,3,4,5,6,7,8].

Then [-1:2:2] will return elements starting from the last element till the third element by printing every second element. i.e. [8,6,4].

6. What are Keywords in Python?

Ans: Keywords in python are reserved words that have special meaning. They are generally used to define type of variables. Keywords cannot be used for variable or function names. There are following keywords in python-

And

Or

Not If

Elif

Else

For

While

Break

Def

7. What are local variables and global variables in Python?

Global Variables:

Variables declared outside a function or in global space are called global variables. These variables can be accessed by any function in the program.

Local Variables:

Any variable declared inside a function is known as a local variable. This variable is present in the local space and not in the global space.

8. Is python case sensitive?

Ans: Yes. Python is a case sensitive language.

9. What is type conversion in Python?

Ans: Type conversion refers to the conversion of one data type into another. int()

– converts any data type into integer type

float() – converts any data type into float type

ord() – converts characters into integer

hex() – converts integers to hexadecimal

10. Is indentation required in python?

Ans: Indentation is necessary for Python. It specifies a block of code. All code within loops, classes, functions, etc is specified within an indented block. It is usually done using four space characters. If your code is not indented necessarily, it will not execute accurately and will throw errors as well.

11. What are functions in Python?

Ans: A function is a block of code which is executed only when it is called. To define a Python function, the def keyword is used.

12. How does break, continue and pass work?

Break

Allows loop termination when some condition is met and the control is transferred to the next statement.

Continue

Allows skipping some part of a loop when some specific condition is met and the control is transferred to the beginning of the loop

Pass

Used when you need some block of code syntactically, but you want to skip its execution. This is basically a null operation. Nothing happens when this is executed

What does [::-1] do?

Ans: [::-1] is used to reverse the order of an array or a sequence.

13. What are python iterators?

Ans: Iterators are objects which can be traversed though or iterated upon.

14. How do you write comments in python?

Ans: Comments in Python start with a # character. However, alternatively at times, commenting is done using docstrings(strings enclosed within triple quotes).

15. What does len() do?

Ans: It is used to determine the length of a string, a list, an array, etc.

Example:

```
str = "abcde"
```

len(str) output:

5

16. What are negative indexes and why are they used?

Ans: The sequences in Python are indexed and it consists of the positive as well as negative numbers. The numbers that are positive uses „0“ that is uses as first index and „1“ as the second index and the process goes on like that.

The index for the negative number starts from „-1“ that represents the last index in the sequence and „-2“ as the penultimate index and the sequence carries forward like the positive number.

The negative index is used to remove any new-line spaces from the string and allow the string to except the last character that is given as S[:-1]. The negative index is also used to show the index to represent the string in correct order.

17. What are Python packages?

Ans: Python packages are namespaces containing multiple modules.

18. How can files be deleted in Python?

Ans: To delete a file in Python, you need to import the OS Module. After that, you need to use the os.remove() function.

Example:

```
import os
```

```
os.remove("xyz.txt")
```

19. What are the built-in types of python?

Ans: Built-in types in Python are as follows –

Integers

Floating-point Complex

numbers Strings

Boolean

20. How to import modules in python?

Modules can be imported using the import keyword. You can import modules in following way:

Example: import calendar

21. Write a program in Python to produce Star triangle.

```
def func(r):  
    for i in range(r):  
        print(" "*(r-i-1)+"*"*((2*i+1)))  
func(9)
```

22. Explain Inheritance in Python with an example.

Ans: Inheritance allows One class to gain all the members(say attributes and methods) of another class. Inheritance provides code reusability, makes it easier to create and maintain an application. The class from which we are inheriting is called super-class and the class that is inherited is called a derived / child class. They are different types of inheritance supported by Python:

1. Single Inheritance – where a derived class acquires the members of a single super class.
2. Multi-level inheritance – a derived class d1 is inherited from base class base1, and d2 are inherited from base2.
3. Hierarchical inheritance – from one base class you can inherit any number of child classes
4. Multiple inheritance – a derived class is inherited from more than one base class.

23. How are classes created in Python?

Ans: Class in Python is created using the class keyword.

Example:

```
class Employee:  
    def __init__(self, name):  
        self.name = name  
E1=Employee("abc")  
print(E1.name)
```

24. Does python support multiple inheritance?

Ans: Multiple inheritance means that a class can be derived from more than one parent classes. Python does support multiple inheritance, unlike Java.

25. What is Polymorphism in Python?

Ans: Polymorphism means the ability to take multiple forms. So, for instance, if the parent class has a method named ABC then the child class also can have a method with the same name ABC having its own parameters and variables. Python allows polymorphism.

26. How do you do data abstraction in Python?

Ans: Data Abstraction is providing only the required details and hiding the implementation from the world. It can be achieved in Python by using interfaces and abstract classes.

27. Does python make use of access specifiers?

Ans: Python does not deprive access to an instance variable or function. Python lays down the concept of prefixing the name of the variable, function or method with a single or double underscore to imitate the behavior of protected and private access specifiers.

28. How to create an empty class in Python?

Ans: An empty class is a class that does not have any code defined within its block. It can be created using the pass keyword. However, you can create objects of this class outside the class itself. IN PYTHON THE PASS command does nothing when its executed. it's a null statement.

For example- class

a:

```
pass
obj=a()
obj.name="xyz"
print("Name = ",obj.name
```

Output:

Name = xyz

29. What does an object() do?

Ans: It returns a featureless object that is a base for all classes. Also, it does not take any parameters.

Appendix-A

This section lists the additional exercise problems from each module(21PSP13/23)which should be designed, implemented and executed by all students in each laboratory session.

	Module 1
	Write a Python program to:
1	WAP to check whether the number is even or odd.
2	WAP to add two numbers. Follow proper rules in creating variables, display the output in a neat manner
3	WAP to ask the user the base and power value and print the resultant exponential value.
4	WAP to checks if the number is positive or negative and display an appropriate message.
5	WAP to illustrate use of simple if statement to compute area of rectangle with positive length and breadth.
6	WAP to illustrate use of simple if statement to compute area of circle for positive radius.
7	WAP to print sum of natural numbers up to.
8	WAP to Print the multiplication table
9	Design a simple weight assistant program that gives us a notification to diet or not to diet: “Raj You Weight 70kgs, You must diet”.
10	A company decided to give bonus of 5% to employee if his/her year of service is more than 5 years. Ask user for their salary and year of service and print the net bonus amount.

	Module 2
	Write a Python program to:
1	WAP to Add Two Numbers using functions
2	WAP to find Factorial of a Number using recursion functions.
3	WAP to Check whether the is Prime Number or not.
4	WAP to Sort Dictionary by Keys
5	WAP to reverse a list
6	WAP to Find Length of a List.
7	WAP to Pop Element by Index.
8	WAP to Convert List to Set to Remove Duplicates.
9	WAP to Count Frequency of Elements.
10	For a given list num=[45,22,14,65,97,72], write a python program to replace all the integers divisible by 3 with “ppp” and all integers divisible by 5 with “qqq” and replace all the integers divisible by both 3 and 5 with “pppqqq” and display the output

	Module 3
	Write a Python program to:
1	WAP to check the given string is palindrome or not.
2	WAP to Reverse a String
3	WAP to Count Vowels in a String
4	WAP to Convert to Lowercase
5	WAP to transpose a matrix using a nested loop
6	WAP to count the total number of vowels, consonants and blanks in a String

Python Programming (BPRLC205B)

7	Write python code to convert roman to integer.
8	WAP to Validate a Phone Number and Validate an Email Address
9	WAP to Remove Punctuations From a String.
10	WAP to create phone number and email address by using regular expression.

Module 4

Write a Python program to:

1	WAP to Read from a File and Append to a File
2	WAP to Open a File in Write Mode
3	WAP to Check if File Exists
4	WAP to Rename a File.
5	WAP to Read and Strip Whitespace from Lines
6	WAP to Count Lines in a File
7	WAP to Get the Absolute Path of a File
8	WAP to Extract a ZIP File with Password
9	WAP to Organize Files by Extension
10	WAP to List All Files in a Directory.

Module 5

Write a Python program to:

1	WAP to Convert temperature from Celsius to Fahrenheit.
2	WAP to find GCD of Two Numbers
3	WAP to find GCD of Two Numbers
4	WAP to Rename Files with American Dates
5	WAP to swap values of two variables using temporary variable
6	WAP for Creating a DataFrame from a Dictionary
7	WAP for Raising an Exception
8	WAP to plot Scatter Plot
9	WAP to plot Bar graph
10	WAP to Creating a DataFrame from a Dictionary